Demo video and Reflection: FairShare – Maral Bat-Erdene

1. **Demo Video:** https://youtu.be/F8LlJvuENdw

The video is private and only accessible through this link on YouTube.

2. **Reflection**

1. Describe your project.

FairShare is a mobile expense-splitting application built for Android. It allows users to create shared expense rooms, invite friends, add bills, assign items to members, and track who owes what in real time. Users can "settle" debts directly using the payment methods saved in the app, choose their preferred currency with live exchange rate conversion, and archive rooms once everything is settled. The app supports capturing the receipt photo and Gemini AI-powered item extraction to make bill entry faster.

2. Did you use GenAI? Which and how?

Yes, I used two AI tools throughout development. Claude Sonnet 4.6 was my primary debugging and refinement tool. I used it to resolve crashes, including a "Failed to convert value of type Long to Date" Firestore deserialization error, threading issues where "Compose" state was being written from background threads, and "mutableStateListOf" index mismatch crashes that occurred when new members were added to a group mid-session. Beyond debugging, I used it to refine screen layouts, write code documentation, restructure the logical flow, and reason through technical decisions throughout the project. Secondly, Google AI Studio, specifically Gemini, powered the receipt intelligence feature. When a user photographs a receipt, Gemini reads the image and automatically extracts every line item by name and price, eliminating manual entry. The extracted items are then displayed in the bill screen where each one can be assigned to the right person with a single tap. This made the "split by item" method genuinely more useful in practice with no more arguments about who ordered what, and no typing out a long restaurant bill.

3. Which technologies did you use?

- **Kotlin** and **Jetpack Compose** for the Android UI
- **Firebase Firestore** for real-time database and authentication
- **Firebase Auth** for user account management
- **Supabase Storage** for receipt image uploads
- **Fixer.io API** for live foreign exchange rates

- **Coil** for async image loading
- **Hilt** for dependency injection
- **Navigation3** for screen navigation
- **Google Gemini API** for AI-powered receipt scanning

4. What did you learn, and what was the biggest challenge?

I learned how critical it is to think carefully about data ownership and consent in multi-user systems. Throughout the invitation flow and settle debt logic, I was deliberate about avoiding design decisions with unintended ethical implications by always asking for explicit user consent when accessing the camera, when accepting a debt payment, and when joining a group. A user should never be added to a financial space or have money moved on their behalf without agreeing to it first.

Also, I deepened my understanding of Firestore's data model, particularly around safe type deserialization and real-time snapshot listeners. The biggest technical challenge was managing Compose state correctly across asynchronous Firestore callbacks and ensuring state writes always happened on the main thread and that remembered lists updated properly when the underlying data changed mid-session.

Looking back, there are two things I wish I had more time to address. First, my color palette of the rose and green scheme I chose could be difficult for users with certain types of color blindness, and I would add multiple accessibility themes given more time. Second, I would expand the notifications system beyond transactional alerts. Rather than only reminding users that they owe money, I'd want the app to occasionally prompt something like "you haven't caught up with this group in a while, so invite them for coffee." Expense tracking can make friendships feel transactional, and I think good design should actively push back against that.

5. What are you most proud of?

I'm most proud of three things. First, the overall user experience. The UI went through a lot of refinement, so now the transitions between screens feel fluid, the visual language is consistent throughout, and feedback from new users suggested the app was intuitive to pick up without any explanation. Getting that level of well-designed front-end took real thoughtfulness and I'm proud of how it turned out.

Second, the currency conversion system. Supporting 15 currencies with live exchange rates required careful arithmetic across the entire data pipeline, where all amounts are stored internally in EUR and converted to the user's preferred currency at display time.

Third, the AI receipt scanning feature. Pointing your camera at a receipt and watching every line item get extracted and laid out for assignment felt genuinely magical when it worked. It turned what would have been the most tedious part of the app, which is entering a long restaurant bill item by item, into something almost effortless.

6. What feedback did you get and how did you address it?

A friend tested the app and said that when they left a group, they didn't want it to just disappear. Shared expense rooms carry memories of a trip to Vienna, a semester living together, a group project, and being able to look back at them felt important to them. That feedback resonated with me immediately. Therefore, in response, I built the Archived Rooms feature. When a user leaves a group, instead of being removed from their home screen entirely, the room moves into a collapsible "Archived Rooms" section. It becomes read-only, so they can still browse the bills, see the final balances, and revisit the history, but they can no longer add bills or interact financially with it. Thus, it helps preserves the memory of the room without cluttering the active space, which I thought was a genuinely thoughtful piece of feedback. Expense apps, like Splitwise, tend to be purely transactional, but the rooms in FairShare often represent real experiences shared with real people, and the design, I tried with full heart, respect that motive as well.

7. How helpful was it to work on project scope and software architecture?

Extremely helpful. Defining the scope early by deciding on expense tracking, not payment processing kept my project extremely focused. The architecture decision to use Firestore's real-time snapshot listeners rather than one-time reads meant the UI always reflected live data without manual refresh logic, which simplified a lot of screen-level code. If I hadn't planned the data model (groups, bills, balances, pendingMemberIds) upfront, refactoring it mid-project would have been very exhausting.

8. Can user-system performance metrics enlighten your project's development?

Yes. Concepts like response time and perceived performance were relevant and Firestore reads are fast but not instant, so adding loading indicators and optimistic UI updates (closing dialogs before the server confirms) made the app feel more responsive. The concept of error recovery was also important, so rather than letting crashes surface to users, snackbars with clear messages kept users informed without breaking the experience. Learnability was a consideration too since the settlement flow needed to be intuitive for users who had never used a bill-splitting app before.

---

9. What possible misuses or ethical issues can you identify?

- **Financial coercion**: a user could create a group, add large fake bills assigned to someone, and pressure them to "settle." The app has no bill dispute mechanism.
- **Privacy**: group members can see each other's balances and spending patterns inside the group-only, which might reveal financial behavior people may want kept private.
- **Currency manipulation**: a malicious user could log bills in a high-value currency to inflate what others owe them over time.

10. How would you describe your project to a recruiter?

FairShare is a full-stack Android application built with Jetpack Compose and Firebase that solves the real-world problem of splitting expenses fairly among groups of people. It features real-time data sync, multi-currency support with live exchange rates, AI-powered receipt scanning, photo storage via Supabase, and a consent-based invitation system. I built it end to end from data modeling and backend transactions to UI design and state management, and debugged production-level issues including threading bugs, Firestore deserialization errors, and race conditions in asynchronous flows.

11. How did you like your demo video?

Overall I was very happy with it. The core logic came through clearly and I managed to cover a lot of ground in about 10 minutes. Narrating while demoing is genuinely harder than it looks and I think I held it together well. However, if I could redo it, I would spend more time on the human side of the app. FairShare is about friendships, trips, and shared memories, and I did not show that warmth as much as I wanted to. I leaned toward demonstrating what I built technically rather than what it feels like to actually use it with friends. That was a conscious choice because I wanted to highlight my development skills, but looking back I think showing both is possible and I would try harder to find that balance.

12. If you were to continue, what would you do next? What is your next side project?

For FairShare, the first thing I would add is push notifications. Right now the app is reactive, you have to open it to see what changed. Real alerts when someone sends you an invite, settles a debt, or adds a new bill would make the experience feel alive. I would also add a bill editing and dispute flow, because right now once a bill is saved it cannot be changed, which is a real limitation. Spending history charts and group

expense categories would round it out nicely and give users a clearer picture of where their money is going over time.

I would also revise the color palette. The rose and green scheme I chose could be difficult for users with certain types of color blindness, and I would add multiple accessibility themes given more time.

The feature I am most excited about adding is friendship notifications. Occasionally prompting something like "you have not caught up with this group in a while, invite them for coffee" would make the app feel like it actually cares about the friendships behind the finances. Expense tracking can make relationships feel transactional, and I think good design should actively push back against that.

For my next project, I want to build a BookTok inspired reading platform as a website. I love reading and I want to make classic and copyright-free books more accessible, modern, and community driven. The idea is to host books with AI powered features that bring them to life, like voice overs where you can assign a unique voice to each character, making it feel like an audiobook cast. There would also be comment sections, discussion posts, and community spaces around each book, so readers can connect the way they do on Wattpad but in one dedicated place built for that purpose. It combines everything I care about, reading, community, and building things that feel genuinely useful and fun to use.